Java OOP Laboratory Exercise: Campus Café Ordering System

ITS 202: Object-Oriented Programming

Submitted by:

Mae-an Acabal

Jella T. Quibot

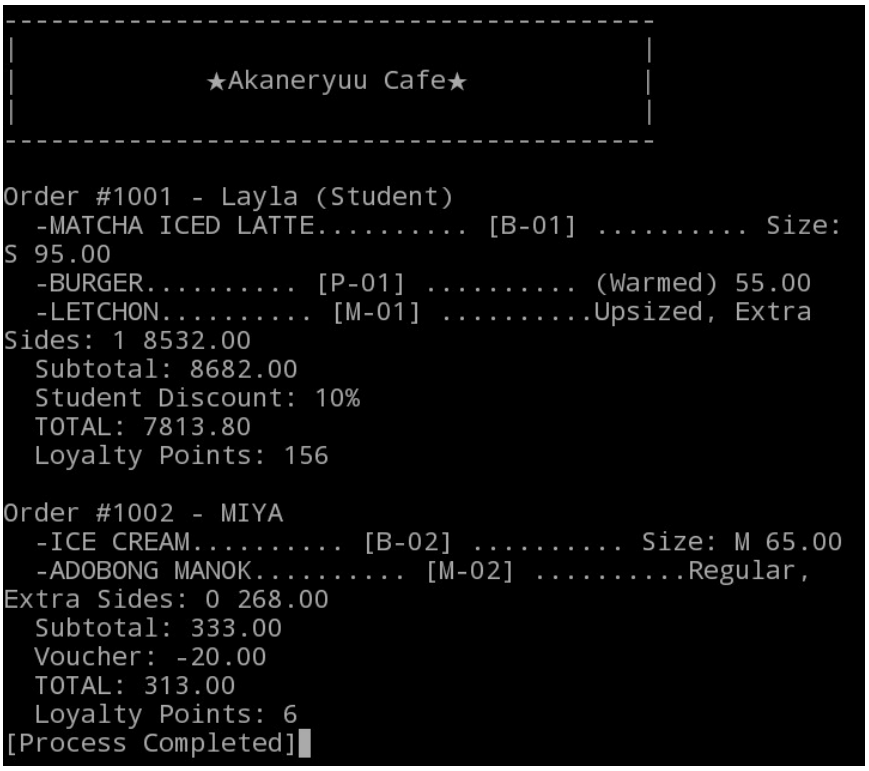
Section: C

NOVEMBER 10, 2026

Concept Explanation

1. **if Statement:** We used if statements where an extra step is needed only in some cases, such as adding a charge when a pastry is warmed or when a meal is upsized, and checking that an order slot is not empty.
2. **if-else Statement:** We used if-else to choose between outcomes: setting a beverage price by size (S, M, L), rejecting negative prices in setBasePrice(), limiting an order to its item limit, and keeping the total from going below zero when a voucher is applied.
3. **Classes and Objects:** The program has several classes (MenuItem, Beverage, Pastry, Meal, Customer, Order, and the main class), each in its own Java file. The objects are created in main, where we show how they work together.
4. **Attributes and Behaviors:** Attributes store each object's data, such as a beverage's size or an item's base price. Behaviors are the methods, such as computePrice(), addItem() and printReceipt().
5. **Object Interaction (Order):** The Order class connects a Customer with its menu items. When the receipt is printed, Order asks each item for its description and price, computes the subtotal and total, and updates the customer's loyalty points.
6. **Java Naming Conventions:** Classes use PascalCase (MenuItem, Beverage), while methods and variables use camelCase (basePrice, computeTotal()). This keeps the code readable and consistent.
7. **Default Constructor:** Default constructors give objects starting values when no information is provided, so an object can still be created in a valid state.
8. **Parameterized Constructor:** Parameterized constructors set an object's details at creation. The subclasses call super(...) to pass the name, price and code up to MenuItem.
9. **Constructor Overloading:** The Customer class has more than one constructor so a customer can be created in different ways: with no information, with only a name, or with a name and student status.
10. **public / private / protected:** Most instance variables are private to protect them. Variables the subclasses need, like name and basePrice, are protected. Methods other classes must call are public.
11. **Getters, Setters, and Data Hiding:** Getters and setters are used instead of public fields. Setters validate input, for example rejecting negative prices and loyalty points.
12. **Method Overloading:** We used methods with the same name but different parameters, such as addPoints(int) and addPoints(double), and computeTotal(int) and computeTotal(double).
13. **Method Overriding:** Beverage, Pastry and Meal each override computePrice() and getDescription() from MenuItem, using the @Override annotation, so each item type calculates its own price and description.
14. **Abstraction:** MenuItem is an abstract class because it is the general parent of all menu items. Its abstract method computePrice() forces every subclass to supply its own pricing rule.

Screenshot / Output



Challenges Encountered

- **Comparing Strings:** We first compared sizes with ==, which did not work for Strings. We fixed it by using .equals() for each comparison.
- **Invalid Size:** When an invalid size was passed to Beverage, the setter rejected it and left the size empty. We set a default size before checking the given input.
- **Abstract Method:** We forgot to declare MenuItem as abstract and gave computePrice() a body. We added the abstract keyword and left only the method declaration.
- **Loyalty Points:** Our addPoints() methods replaced the existing points instead of adding to them. We fixed this by validating the input first, then using +=.
- **Wrong Output Values:** Some prices and loyalty points in our first run looked far too large. We traced the calculations in the item and order classes and corrected them.